

UNIT 3

Structured Query Language

Introduction to SQL

- ✓ SQL is a standard database language used to access and manipulate data in databases.
- ✓ The language, Structured English Query Language (SEQUEL) was developed by IBM Corporation, Inc., to use Codd's model. SEQUEL later became SQL (still pronounced "sequel").
- ✓ SQL stands for Structured Query Language.
- ✓ In 1979, Relational Software, Inc. (now Oracle) introduced the first commercially available implementation of SQL. Today, SQL is accepted as the standard RDBMS language.
- ✓ By executing queries SQL can create, update, delete, and retrieve data in within a database management system like MySQL, Oracle, PostgreSQL, etc.
- ✓ Overall SQL is a query language that communicates with databases.

The SQL language has several parts,

Data Definition Language (DDL): SQL provides a set of commands to define and modify the structure of a database, including creating tables, modifying table structure, and dropping tables.

Data Manipulation Language (DML): SQL provides a set of commands to manipulate data within a database, including adding, modifying, and deleting data.

SQL provides a rich set of commands for querying a database to retrieve data, including the ability to filter, sort, group, and join data from multiple tables.

Transaction Control: SQL supports transaction processing, which allows users to group a set of database operations into a single transaction that can be rolled back in case of failure.

Data Integrity: SQL includes features to enforce data integrity, such as the ability to specify constraints on the values that can be inserted or updated in a table, and to enforce referential integrity between tables.

User Access Control: SQL provides mechanisms to control user access to a database, including the ability to grant and revoke privileges to perform certain operations on the database.

Data types in SQL

char(n) :Fixed length character string, with user-specified length n.

varchar(n) : Variable length character strings, with user-specified maximum length n.

int : Integer (a finite subset of the integers that is machine-dependent).

smallint:Small integer (a machine-dependent subset of the integer domain type).

numeric(p,d):Fixed point number, with user-specified precision of p digits, with d digits to the right of decimal point. (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)

real, double precision: Floating point and double-precision floating point numbers, with machine-dependent precision.

float(n): Floating point number, with user-specified precision of at least n digits.

Date and Time types in SQL

date: A calendar date containing a (four-digit) year, month, and day of the month.

time: The time of day, in hours, minutes and seconds.

timestamp: A combination of date and time

Date and time values can be specified like this:

date: '2001-04-25'

time:'09:30:00'

timestamp '2001-04-25 10:29:01.45'

DDL and DML statements in SQL

Data Definition Language (DDL)

- ✓ The Data Definition Language is made up of SQL commands that can be used to design the database structure.
- ✓ DDL refers to a set of SQL instructions for creating, modifying, and deleting database structures, but not data
- ✓ Popular DDL commands are: CREATE, DROP, ALTER and TRUNCATE.

CREATE: The database or its objects are created with this command (like table, views, store procedure, and triggers).

- ✓ A database is a systematic collection of data. To store data in a well-structured manner, the first step with SQL is to establish a database. To build a new database in SQL, use the CREATE DATABASE statement.

Syntax: CREATE DATABASE db_name;

Example:

CREATE DATABASE student_db;

The above example will create a database named student_db;

- ✓ We've already learned how to create databases. To save the information, we'll need a table. In SQL, the CREATE TABLE statement is used to make a table. A table is made up of rows and columns, as we all know. As a result, while constructing tables, we must give SQL all relevant information, such as the names of the columns, the type of data to be stored in the columns, the data size, and so on.

Syntax:

CREATE TABLE table_name(

column1 data_type1,

column2 data_type2,

column3 data_type3,

column4 data_type4,

.....

);

Example:

```
CREATE TABLE student_info(  
sid int,  
name varchar(30),  
program varchar(30),  
roll int);
```

The above command will create the table schema that look like:

sid	name	program	roll
-----	------	---------	------

DROP

- ✓ The DROP statement deletes existing database objects such as tables and views.

For dropping table

Syntax: DROP TABLE table_name;

Example: DROP TABLE student_info;

For dropping database

Syntax: DROP DATABASE db_name;

Example: DROP DATABASE student_db;

ALTER

- ✓ The ALTER statement in SQL is used to make changes to the structure of existing database objects. It allows you to modify tables, views, and other database elements.

Example:

- In an existing table, this command is used to add, delete or edit columns.
- It can also be used to create and remove constraints from a table that already exists.

To add Column in table

Syntax:

ALTER TABLE table_name

ADD column_name datatype;

Example:

ALTER TABLE student_info;

ADD address varchar(30);

To remove existing column from table

Syntax:

ALTER TABLE table_name

DROP COLUMN column_name;

Example:

```
ALTER TABLE student_info  
DROP COLUMN roll;
```

To rename column of table**Syntax:**

```
ALTER TABLE table_name  
CHANGE COLUMN old_name new_name datatype;
```

Example:

```
ALTER TABLE student_info  
CHANGE COLUMN address location varchar(30);
```

(Note: This syntax is for MariaDB and may vary upon different DBMS)

To modify data type of column**Syntax:**

```
ALTER TABLE table_name  
MODIFY column_name datatype;
```

Example:

```
ALTER TABLE student_info  
MODIFY program char(20);
```

TRUNCATE

- ✓ This statement deletes all the rows from the table.
- ✓ This is different from the DROP command, the DROP command deletes the entire table along with the table schema, however TRUNCATE just deletes all the rows and leaves an empty table.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE student_info;
```

//Deletes all the rows from the table student_info.

By performing all the above operations, finally our table named student_info becomes

Sid	Name	program	location
int	varchar(30)	char(20)	varchar(30)

Data Manipulation Language (DML)

- ✓ The SQL commands that deal with manipulating data in a database are classified as DML (Data Manipulation Language)
- ✓ The popular commands that come under DML are INSERT, UPDATE, DELETE, SELECT

INSERT

- ✓ This command is used to insert records in a table.
- ✓ When you are not inserting the data for all the columns and leaving some columns empty. In that case specify the column name and corresponding value. The non selected field will have NULL value inserted upon execution of the given query.

Syntax:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO student_info(sid,name,location)  
VALUES(1,'Hari','Pokhara');
```

- ✓ When inserting the data for all the columns. No need to specify column name.

Syntax:

```
INSERT INTO table_name  
  
VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO student_info  
  
VALUES(2,'Rita','Computer','Butwal');
```

UPDATE

- ✓ In SQL, the UPDATE statement is used to update data in an existing database table.

Syntax:

```
UPDATE table_name  
  
SET column1 = value1, column2 = value2,...  
  
WHERE condition;
```

Example:

```
UPDATE student_info  
SET location='kathmandu'  
WHERE sid=2;
```

DELETE

- ✓ DELETE statement is used to delete records from a table.
- ✓ Depending on the condition we set in the WHERE clause, we can delete a single record or numerous records.

Syntax:

DELETE FROM table_name

WHERE condition;

Example1:

```
DELETE FROM student_info  
WHERE location='kathmandu';
```

//Delete records of student from table named student_info whose location is Kathmandu

Example 2:

```
DELETE FROM student_info;
```

//Delete all records from table named student_info;

SELECT

- ✓ SELECT command fetches the records from the specified table that matches the given condition, if no condition is provided, it fetches all the records from the table.

Syntax:

SELECT column1, column2, ...

FROM table_name;

Here, column1, column2, ... are the column names of the table we want to select data from.

- ✓ If we want to apply conditions while selecting the data then syntax becomes

SELECT column1, column2, ...

FROM table_name

WHERE condition;

- ✓ If we want to select all the columns and rows available in the table, use the following syntax:

```
SELECT * FROM table_name;
```

Example:

```
SELECT * FROM student_info;
```

//Displays all the information of students from table named student_info

```
SELECT name,program
```

```
FROM student_info
```

```
WHERE location='pokhara';
```

//Displays name and program of students from table named student_info whose location is 'pokhara'

Note: Some authors grouped SELECT command as DQL(Data Query Language)

SQL constraints

- SQL constraints are used to specify rules for the data in a table.
- Constraints are used to limit the type of data that can go into a table.
- This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

The following constraints are commonly used in SQL:

NOT NULL - Ensures that a column cannot have a NULL value

UNIQUE - Ensures that all values in a column are different

PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table

FOREIGN KEY - Prevents actions that would destroy links between tables

CHECK - Ensures that the values in a column satisfies a specific condition

DEFAULT - Sets a default value for a column if no value is specified

NOT NULL

- ✓ By default, a column can hold NULL values.
- ✓ The NOT NULL constraint enforces a column to NOT accept NULL values.
- ✓ This enforces a field to always contain a value, which means that you cannot insert a new record without adding a value to this field.

❖ create table with NOT NULL constraint

Example:

```
CREATE TABLE Colleges (  
college_id INT NOT NULL,  
college_code VARCHAR(20),  
college_name VARCHAR(50)  
);
```

❖ Add the NOT NULL constraint to a column in an existing table

Example:

```
ALTER TABLE Colleges  
MODIFY COLUMN college_id INT NOT NULL;
```

❖ Remove NOT NULL Constraint

Example:

```
ALTER TABLE Colleges  
MODIFY college_id INT;
```

UNIQUE

- ✓ The UNIQUE constraint ensures that all values in a column are different.

❖ Create a table with unique constraint

Example

```
CREATE TABLE Colleges (  
college_id INT NOT NULL UNIQUE,  
college_code VARCHAR(20) UNIQUE,  
college_name VARCHAR(50)  
);
```

❖ Add the UNIQUE constraint to an existing column

For single column

Example

```
ALTER TABLE Colleges  
ADD UNIQUE (college_id);
```

For multiple columns

Example

```
ALTER TABLE Colleges  
ADD UNIQUE Unique_College (college_id, college_code);
```

- ✓ Here, the SQL command adds the UNIQUE constraint to college_id and college_code columns in the existing Colleges table.
- ✓ Also, Unique_College is a name given to the UNIQUE constraint defined for college_id and college_code columns.

❖ DROP a UNIQUE Constraint

Example

```
ALTER TABLE Colleges  
DROP INDEX Unique_College;
```

PRIMARY KEY

- ✓ The PRIMARY KEY constraint uniquely identifies each record in a table.
- ✓ Primary keys must contain UNIQUE values, and cannot contain NULL values.

❖ Create table with PRIMARY KEY constraint

Syntax:

```
CREATE TABLE table_name (  
column1 data_type,  
.....,  
[CONSTRAINT constraint_name] PRIMARY KEY (column1)  
);
```

Example

```
CREATE TABLE Colleges (  
college_id INT,  
college_code VARCHAR(20) ,  
college_name VARCHAR(50),  
CONSTRAINT CollegePK PRIMARY KEY (college_id)  
);
```

//Create Colleges table with primary key college_id

❖ Add the PRIMARY KEY constraint to a column in an existing table

Example

```
ALTER TABLE Colleges  
ADD CONSTRAINT CollegePK PRIMARY KEY (college_id);
```

❖ DROP a PRIMARY KEY Constraint

Example

```
ALTER TABLE Colleges  
DROP PRIMARY KEY;
```

DEFAULT

- ✓ the DEFAULT constraint is used to set a default value if we try to insert an empty value into a column.
- ✓ However if the user provides value then the particular value will be stored.

❖ Default constraint while creating table

The following example set default value of college_country column to 'Nepal'

Example:

```
CREATE TABLE Colleges (  
    college_id INT PRIMARY KEY,  
    college_code VARCHAR(20),  
    college_country VARCHAR(20) DEFAULT 'Nepal'  
);
```

❖ Add the DEFAULT constraint to an existing column

Example:

```
ALTER TABLE Colleges  
ALTER college_country SET DEFAULT 'Nepal';
```

❖ Remove DEFAULT Constraint

Example:

```
ALTER TABLE Colleges  
ALTER college_country DROP DEFAULT;
```

CHECK

- ✓ The CHECK constraint is used to limit the value range that can be placed in a column.
- ✓ If you define a CHECK constraint on a column it will allow only certain values for this column.

❖ CHECK constraint while creating table

Example:

Here we are Applying the CHECK constraint named amountCK the constraint makes sure that amount is greater than 0.

```
CREATE TABLE Orders (  
order_id INT PRIMARY KEY,  
amount INT,  
CONSTRAINT amountCK CHECK (amount > 0)  
);
```

❖ Add CHECK Constraint in Existing Table

Here we add CHECK constraint named amountCK the constraint makes sure that amount is greater than 0.

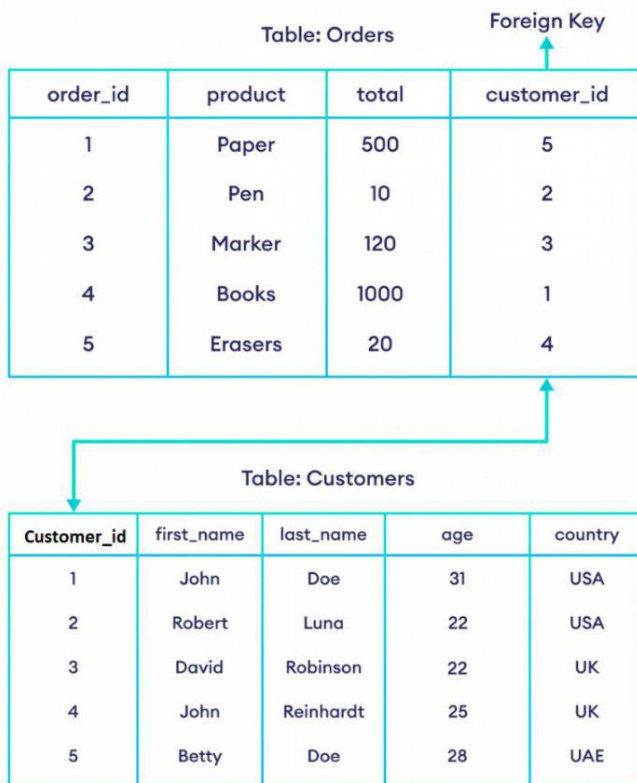
```
ALTER TABLE Orders  
ADD CONSTRAINT amountCK CHECK (amount > 0);
```

❖ Remove CHECK Constraint

```
ALTER TABLE Orders  
DROP CONSTRAINT amountCK;
```

FOREIGN KEY

The FOREIGN KEY constraint in SQL establishes a relationship between two tables by linking columns in one table to those in another.



- ✓ Here, the **customer_id** field in the Orders table is a FOREIGN KEY that references the **customer_id** field in the Customers table.
- ✓ This means that the value of the **customer_id** (of the Orders table) must be a value from the **customer_id** column (of the Customers table).

The syntax of the SQL FOREIGN KEY constraint is:

```
CREATE TABLE table_name (
    column1 data_type,
    column2 data_type,
    .....
    [CONSTRAINT CONSTRAINT_NAME] FOREIGN KEY (column_name)
    REFERENCES referenced_table_name (referenced_column_name)
);
```

Here,

- ✓ table_name is the name of the table where the FOREIGN KEY constraint is to be defined
- ✓ column_name is the name of the column where the FOREIGN KEY constraint is to be defined
- ✓ referenced_table_name and referenced_column_name are the names of the table and the column that the FOREIGN KEY constraint references
- ✓ [CONSTRAINT CONSTRAINT_NAME] is optional

Let us see with following example

- ✓ This table doesn't have a foreign key
- ✓ add foreign key to the customer_id field
- ✓ the foreign key references the id field of the Customers table

-- this table doesn't have a foreign key

```
CREATE TABLE Customers (  
  customer_id INT,  
  first_name VARCHAR(40),  
  last_name VARCHAR(40),  
  age INT,  
  country VARCHAR(10),  
  CONSTRAINT CustomersPK PRIMARY KEY (customer_id)  
);  
  
-- add foreign key to the customer_id field  
-- the foreign key references the id field of the Customers table  
CREATE TABLE Orders (  
  order_id INT,  
  product VARCHAR(40),  
  total INT,  
  customer_id INT,  
  CONSTRAINT OrdersPK PRIMARY KEY (order_id),  
  CONSTRAINT CustomerOrdersFK FOREIGN KEY (customer_id) REFERENCES  
Customers(customer_id)  
);
```

Add the FOREIGN KEY constraint to an existing table

- ✓ add foreign key to the **customer_id** field of Orders the foreign key references the **customer_id** field of Customers

```
ALTER TABLE Orders  
ADD FOREIGN KEY (customer_id) REFERENCES Customers(customer_id);
```

Remove a FOREIGN KEY Constraint

```
ALTER TABLE Orders  
DROP FOREIGN KEY CustomerOrdersFK;
```

Operators in SQL

- An operator is a reserved word or a character that is used to query our database in a SQL expression.
- To query a database using operators, we use a WHERE clause.
- The operator manipulates the data and gives the result based on the operator's functionality.

Before starting with operators let us consider the following relation that we use to illustrate the examples of operators

Customers(customer_id,first_name,last_name,age,country);

Orders(order_id,product,total,customer_id);

Some operators available in SQL are:

Arithmetic Operators

- ✓ These operators are used to perform operations such as addition, multiplication, subtraction etc.
- ✓ Example. + (Addition), - (subtraction), * (multiplication), / (division), % (modulus) etc.

```
UPDATE Orders
SET total=total+15;
```

This query increase the total amount of all records by 15.

Comparison Operators

- ✓ We can compare two values using comparison operators in SQL.
- ✓ These operators return either 1 (means true) or 0 (means false).

Example:

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<> , !=	Not equal to

```
SELECT *
FROM customers
WHERE age>20;
```

This query display the information of customers whose age is greater than 20

Logical operators

We can use logical operators to compare multiple SQL commands. These operators return either 1 (means true) or 0 (means false).

Some of the Logical operators available in SQL are,

AND
OR
NOT
BETWEEN
IN
LIKE

AND

- ❖ Returns the records if all the conditions separated by AND are TRUE

Example:

- ✓ *Display the first_name and last_name of all customers who live in 'Nepal' and have the last_name 'Paudel'*

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'Nepal' AND last_name = 'Paudel';
```

OR

- ❖ Returns the records for which any of the conditions separated by OR is true

Example:

- ✓ *Display the first_name and last_name of all customers who either live in the 'Nepal' or have the last name 'Paudel'*

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'Nepal' OR last_name = 'Paudel';
```

NOT

- ❖ Used to reverse the output of any logical operator

Example:

Display customers who don't live in the USA

```
SELECT first_name, last_name  
FROM Customers  
WHERE NOT country = 'USA';
```

Combining Multiple Operators

- ✓ It is also possible to combine multiple AND, OR and NOT operators in an SQL statement.

Display customers who live in either USA or UK and whose age is less than 26

```
SELECT *  
FROM Customers  
WHERE (country = 'USA' OR country = 'UK') AND age < 26;
```

BETWEEN

- ❖ Returns the rows for which the value lies between the mentioned range.

Example:

Displays customers first_name, last_name, age from customers table whose age lies in the range 20-30

```
SELECT first_name, last_name, age  
FROM Customers  
WHERE age BETWEEN 20 AND 30;
```

Note: The **NOT BETWEEN** operator is used to exclude the rows that match the values in the range. It returns all the rows except the excluded rows.

IN

- ❖ Used to compare a value to a specified value in a list
- ❖ The IN operator selects values that match any one values given in the list

Example:

Select rows if the country lies in following list USA, UK, Nepal, India, Pakistan

```
SELECT *  
FROM Customers  
WHERE country IN ('USA', 'UK', 'Nepal', 'India', 'Pakistan');
```

Note: The **NOT IN** operator is used to exclude the rows that match values in the list. It returns all the rows except the excluded rows

LIKE

- ✓ The SQL LIKE operator is used with the WHERE clause to get a result set that matches the given string pattern.
- ✓ The pattern includes combination of wildcard characters and regular characters

Example :

```
SELECT *  
FROM Customers  
WHERE last_name LIKE 'r%';
```

- ✓ Here, % (means zero or more characters) is a wildcard character.
- ✓ Hence, the SQL command selects customers whose last_name starts with r followed by zero or more characters after it.

Wildcard Characters

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue,
_	Represents a single character	h_t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
^	Represents any character not in the brackets	h[^oa]t finds hit but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt

Here are some examples showing different LIKE operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE first_name LIKE 'a%'	Finds first_name that starts with "a"
WHERE first_name LIKE '%a'	Finds first_name that ends with "a"
WHERE first_name LIKE '%or%'	Finds first_name that have "or" in any position
WHERE first_name LIKE '_r%'	Finds first_name that have "r" in the second position
WHERE first_name LIKE 'a__%'	Finds first_name that starts with "a" and are at least 3 characters in length
WHERE first_name LIKE 'a%h'	Finds first_name that starts with "a" and ends with "h"

NULL values

- ✓ The term NULL in SQL is used to specify that a data value does not exist in the database.
- ✓ If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.

Some common reasons why a value may be NULL

- ❖ The value may not be provided during the data entry.
- ❖ The value is not yet known.
- ✓ It is not possible to test for NULL values with comparison operators, such as =, <, or <>.
- ✓ We will have to use the IS NULL and IS NOT NULL operators instead.

IS NULL

The IS NULL operator is used to test for empty values (NULL values).

Syntax:

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NULL;
```

Example:

The following SQL lists first_name and last_name of all customers with a NULL value in the "country" field

```
SELECT first_name,last_name  
FROM Customers  
WHERE country IS NULL;
```

IS NOT NULL

The IS NOT NULL operator is used to test for non-empty values (NOT NULL values).

Syntax

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NOT NULL;
```

The following SQL lists first_name,last_name and country of all customers with a value in the "country " field:

```
SELECT first_name,last_name,country  
FROM Customers  
WHERE country IS NOT NULL;
```

SQL SELECT DISTINCT

- ✓ The SQL SELECT DISTINCT statement retrieves distinct values from a database table.

Example 1:

Select the unique ages from the Customers table

```
SELECT DISTINCT age
FROM Customers;
```

Example 2:

- ✓ select the unique countries from the customers table

```
SELECT DISTINCT country
FROM Customers;
```

SQL DISTINCT With Multiple Columns

- ✓ We can also use SELECT DISTINCT with multiple columns.

Select rows if the first name and country of a customer is unique

```
SELECT DISTINCT country, first_name
FROM Customers;
```

Rename operation

- ✓ The AS command is used to rename a column or table with an alias.
- ✓ An alias only exists for the duration of the query.
- ✓ We can also use aliases with more than one column.

Example1:

```
SELECT first_name AS name
FROM Customers;
```

Here, the SQL command selects the first_name column of Customers. However, the column name will change to name in the result set.

Example2:

```
SELECT customer_id AS cid, first_name AS name
FROM Customers;
```

Here, the SQL command selects customer_id as cid and first_name as name

Sorting Results

- ✓ The ORDER BY keyword is used to sort the result-set in ascending or descending order.
- ✓ The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

ORDER BY Syntax

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

Example:

The following SQL statement selects all customers from the "Customers" table, sorted by the "Country" column:

```
SELECT *  
FROM Customers  
ORDER BY country;
```

The following SQL statement selects all customers from the "Customers" table, sorted DESCENDING by the "Country" column.

```
SELECT * FROM  
Customers  
ORDER BY country DESC;
```

ORDER BY Several Columns

The following SQL statement selects all customers from the "Customers" table, sorted by the "country" and the "first_name" column. This means that it orders by Country, but if some rows have the same Country, it orders them by first_name:

Example

```
SELECT * FROM Customers  
ORDER BY country,first_name;
```

The following SQL statement selects all customers from the "Customers" table, sorted ascending by the "Country" and descending by the "first_name" column:

Example

```
SELECT * FROM Customers  
ORDER BY country ASC, first_name DESC;
```

Aggregate functions

An aggregate function in SQL returns one value after calculating multiple values of a column

Let us consider the following relation

```
Employee(employee_id,name,deparment,position,salary);
```

COUNT()

- ✓ The COUNT() function returns the number of rows that matches a specified criterion.

Syntax:

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

Example:

```
SELECT COUNT( DISTINCT employee_id)
FROM Employee;
```

AVG()

- ✓ The AVG() function returns the average value of a numeric column.

Syntax:

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

Example:

```
SELECT AVG(salary)
FROM Employee;
```

SUM()

- ✓ The SUM() function returns the total sum of a numeric column.

Syntax:

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

Example:

```
SELECT SUM(salary)
FROM Employee;
```

MIN()

- ✓ The MIN() function returns the smallest value of the selected column

Syntax:

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

Example:

```
SELECT MIN(salary)
FROM Employee;
```

MAX()

- ✓ The MAX() function returns the largest value of the selected column

Syntax:

```
SELECT MAX(column_name)
FROM table_name
WHERE condition;
```

Example:

```
SELECT MAX(salary)
FROM Employee;
```

GROUP BY and HAVING clause

GROUP BY

- ✓ The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of Employees in each department".
- ✓ The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

Let us consider the following table

Table: Employee

eid	name	address	dept_name	salary
1	Hari	Butwal	Civil	62000
2	Shyam	Kathmandu	Computer	80000
3	Sita	Pokhara	Civil	90000
4	Ramesh	Kathmandu	IT	32000
5	Riya	Pokhara	Computer	76000
6	Dinesh	Kathmandu	Civil	94000
7	Srijana	Butwal	IT	68000

As an illustration consider the query “find the average salary of employee in each department”

```
SELECT dept_name, avg(salary) as average_salary  
FROM Employee  
GROUP BY dept_name;
```

dept_name	average_salary
Civil	82000
Computer	78000
IT	50000

HAVING clause

- ✓ SQL HAVING clause is similar to the WHERE clause; they are both used to filter rows in a table based on conditions.
- ✓ However, the HAVING clause was included in SQL to filter grouped rows instead of single rows.
- ✓ These rows are grouped together by the GROUP BY clause, so, the HAVING clause must always be followed by the GROUP BY clause.
- ✓ It can be used with aggregate functions, whereas the WHERE clause cannot.

As an illustration consider the query “find the name department where the average salary is greater than 60000”

```
SELECT dept_name, avg(salary) as average_salary
FROM employee
GROUP BY dept_name
HAVING avg(salary)>60000;
```

dept_name	average_salary
Civil	82000
Computer	78000

HAVING clause vs WHERE clause

HAVING clause	WHERE clause
HAVING Clause is used to filter record from the groups based on the specified condition.	WHERE Clause is used to filter the records from the table based on the specified condition.
HAVING Clause cannot be used without GROUP BY Clause	WHERE Clause can be used without GROUP BY Clause
HAVING Clause can contain aggregate function	WHERE Clause cannot contain aggregate function
HAVING Clause can only be used with SELECT statement	WHERE Clause can be used with SELECT, UPDATE, DELETE statement.
HAVING Clause implements in column operation	WHERE Clause implements in row operations

Let's take a look at another example, for following relations

```
Customers(customer_id,first_name,last_name,country)
Orders(order_id,product,amount,customer_id)
```

We can write a WHERE clause to filter out rows where the value of amount in the Orders table is less than 500:

```
SELECT customer_id, amount
FROM Orders
WHERE amount < 500;
```

But with the HAVING clause, we can use an aggregate function like SUM to calculate the sum of amounts in the Orders table and get the total order value of less than 500 for each customer:

```
SELECT customer_id, SUM(amount) AS total
FROM Orders
GROUP BY customer_id
HAVING SUM(amount) < 500;
```

Sub Query (Inner Query/Nested Query)

- ✓ A Subquery or Inner query or a Nested query is a query within another SQL query and embedded within clauses, most commonly in the WHERE clause.
- ✓ It is used to return data from a table, and this data will be used in the main query as a condition to further restrict the data to be retrieved.
- ✓ Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN etc.

Consider the following relation

```
employee(emp_id,name,age,department,salary)
```

Subqueries with SELECT statement

Display information of employee whose salary is greater than average salary of all employees

```
select *  
from employee  
where salary>(select avg(salary) from employee);
```

Display the information of employees whose salary is greater than 26000

```
select *  
from employee  
where emp_id in (select emp_id from employee where salary>26000);
```

Display information of employee whose salary is greater than at least one employee of IT department.

```
select *  
from employee  
where salary>some(select salary from employee where department ='IT ');
```

Display information of employee whose salary is greater than that of all employee of IT department.

```
select *  
from employee  
where salary>all(select salary from employee where department ='IT ');
```

Subqueries with UPDATE statement

Increase salary of employees by 10% whose salary is greater than the average salary of all employees.

```
update employee
set salary=salary*1.1
where salary> (select avg(salary) from employee);
```

Subqueries with DELETE statement

Delete the information of employees whose salary is less than average salary of all employees

```
delete from employee where salary < (select avg(salary) from employee);
```

Subqueries with INSERT statement

The INSERT statement uses the data returned from the subquery to insert into another table.

Suppose we want to make each employee board member of company whose department is 'finance' and age>55

Consider a table Boardmember with similar structure as Employee table. Now to copy the records of employee table whose department is 'finance' and age>55 into the Boardmember table, we can use the following syntax.

```
insert into Boardmember
select *
from employee
where emp_id in (select emp_id from employee where department='finance' and age>55);
```

Set operations

- ✓ In SQL, set operation is used to combine the two or more SQL SELECT statements.
- ✓ They allow the results of multiple SELECT queries to be combined into single result set.
- ✓ SQL set operators enable the comparison of rows from multiple tables or a combination of results from multiple queries
- ✓ There are certain rules which must be followed to perform operations using SET operators in SQL. Rules are as follows:
 - ☞ The number of columns in the SELECT statement on which you want to apply the SQL set operators must be the same.
 - ☞ The order of columns must be in the same order.
 - ☞ The selected columns must have the same data type.

UNION

- ✓ The SQL Union operation is used to combine the result of two or more SQL SELECT queries.
- ✓ The union operation eliminates the duplicate rows from its result set.

Syntax:

```
SELECT column_name(s) FROM table_1  
UNION  
SELECT column_name(s) FROM table_2;
```

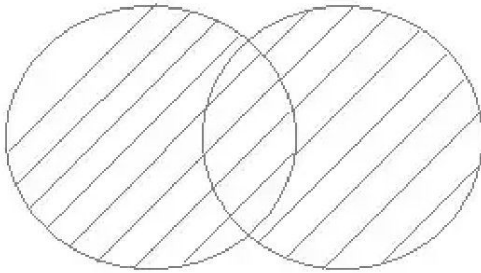


figure: pictorial representation of UNION operation

Let us consider the following two relations.

tbl_first

id	name
1	riya
2	durga
3	anish

tbl_second

Id	name
3	anish
4	roshan
5	rojina

Example:

```
SELECT * FROM tbl_first  
UNION  
SELECT * FROM tbl_second;
```

Output:

Id	name
1	riya
2	durga
3	anish
4	roshan
5	rojina

UNION ALL

- ✓ It is similar to Union but it also shows the duplicate rows.

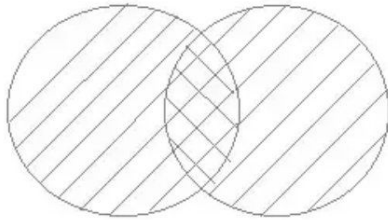


figure: Pictorial representation of UNION ALL operation

Let us consider the following two relations.

tbl_first

Id	name
1	riya
2	durga
3	anish

tbl_second

id	name
3	anish
4	roshan
5	rojina

Example:

```
SELECT * FROM tbl_first
UNION ALL
SELECT * FROM tbl_second;
```

Output

Id	name
1	riya
2	durga
3	anish
3	anish
4	roshan
5	rojina

INTERSECT

- ✓ The Intersect operation returns the common rows from both the SELECT statements.
- ✓ It has no duplicates and it arranges the data in ascending order by default.

Syntax:

```
SELECT column_name(s) FROM table_1  
INTERSECT  
SELECT column_name(s) FROM table_2;
```

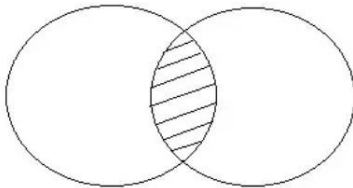


figure: pictorial representation of INTERSECT operation

Let us consider the following two relations.

tbl_first

id	name
1	riya
2	durga
3	anish

tbl_second

id	name
3	anish
4	roshan
5	rojina

Example:

```
SELECT * FROM tbl_first  
INTERSECT  
SELECT * FROM tbl_second;
```

Output

id	name
3	anish

EXCEPT

- ✓ EXCEPT operator is used to display the rows which are present in the first query but absent in the second query.
- ✓ It has no duplicates and data arranged in ascending order by default.

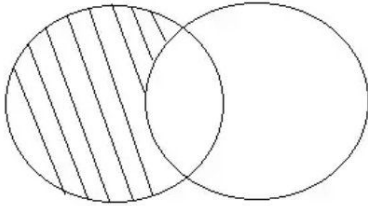


figure: pictorial representation of EXCEPT operation

Syntax:

```
SELECT column_name(s) FROM table_1  
EXCEPT  
SELECT column_name(s) FROM table_2;
```

Let us consider the following two relations.

tbl_first

id	name
1	riya
2	durga
3	anish

tbl_second

id	name
3	anish
4	roshan
5	rojina

Example:

```
SELECT * FROM tbl_first  
EXCEPT  
SELECT * FROM tbl_second;
```

Output

id	name
1	riya
2	durga

Join

- ✓ In SQL, a join is an operation that combines rows from two or more tables based on a related column between them.
- ✓ It allows you to retrieve data from multiple tables simultaneously by establishing a relationship between them.
- ✓ Joins are typically performed using the JOIN keyword in an SQL query.

There are different types of joins that can be used:

Before performing join operations, let us consider the following table

Employee

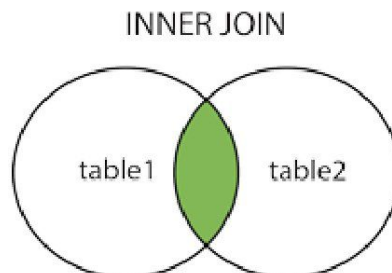
emp_id	emp_name	salary	dept_id
1	anish	25000	1
2	sita	55000	2
3	ronit	40000	4
4	riya	50000	5

Department

dept_id	dept_name
1	sales
2	marketing
3	finance
4	operations

Inner Join

- ✓ Returns only the rows that have matching values in both tables. It combines rows from the tables based on the specified join condition.
- ✓ It is the simple and most popular form of join and assumes as a default join.
- ✓ If we omit the INNER keyword with the JOIN query, we will get the same output.



Syntax:

```
SELECT column_name(s)
FROM table1 INNER JOIN table2
ON
table1.column_name = table2.column_name;
```

Example 1:

```
SELECT *
FROM Employee INNER JOIN Department
ON
Employee.dept_id=Department.dept_id;
```

Output:

emp_id	emp_name	salary	dept_id	dept_id	dept_name
1	anish	25000	1	1	sales
2	sita	55000	2	2	marketing
3	ronit	40000	4	4	operations

Example 2:

```
SELECT Employee.emp_name,Department.dept_name
FROM Employee INNER JOIN Department
ON
Employee.dept_id=Department.dept_id;
```

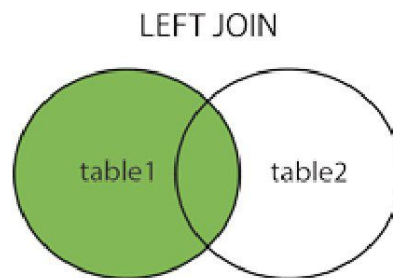
Output:

emp_name	dept_name
Anish	sales
Sita	marketing
ronit	operations

Outer JOIN

- ✓ An outer join is a type of join operation in SQL that includes unmatched rows from one or both tables in the join result.
 - ✓ Unlike an inner join, which only returns matching rows, an outer join ensures that all rows from one table (or both tables) are included in the result set, even if there is no corresponding match in the other table.
- There are three types of outer joins:

1) LEFT JOIN (LEFT OUTER JOIN): Returns all rows from the left table and the matching rows from the right table. If there are no matching rows in the right table, NULL values are returned for the columns of the right table.



Syntax:

```
SELECT column_name(s)
FROM table1 LEFT JOIN table2
ON
table1.column_name = table2.column_name;
```

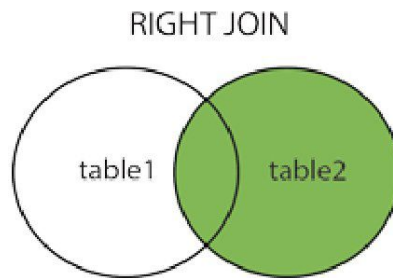
Example:

```
SELECT *
FROM Employee LEFT JOIN Department
ON
Employee.dept_id=Department.dept_id;
```

Output:

emp_id	emp_name	salary	dept_id	dept_id	dept_name
1	anish	25000	1	1	sales
2	sita	55000	2	2	marketing
3	ronit	40000	4	4	operations
4	riya	50000	5	NULL	NULL

2) RIGHT JOIN (RIGHT OUTER JOIN): Returns all rows from the right table and the matching rows from the left table. If there are no matching rows in the left table, NULL values are returned for the columns of the left table.



Syntax:

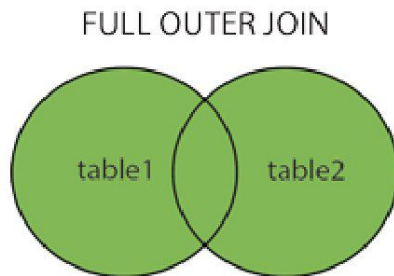
```
SELECT column_name(s)
FROM table1 RIGHT JOIN table2
ON
table1.column_name = table2.column_name;
```

Example:

```
SELECT *
FROM Employee RIGHT JOIN Department
ON
Employee.dept_id=Department.dept_id;
```

emp_id	emp_name	salary	dept_id	dept_id	dept_name
1	anish	25000	1	1	sales
2	sita	55000	2	2	marketing
NULL	NULL	NULL	NULL	3	finance
3	ronit	40000	4	4	operations

3)FULL JOIN (FULL OUTER JOIN): Returns all rows from both tables, regardless of whether they have a match or not. If there is no match, NULL values are returned for the columns of the table that does not have a match.



Syntax:

```
SELECT column_name(s)
FROM table1 FULL JOIN table2
ON
table1.column_name = table2.column_name;
```

Example:

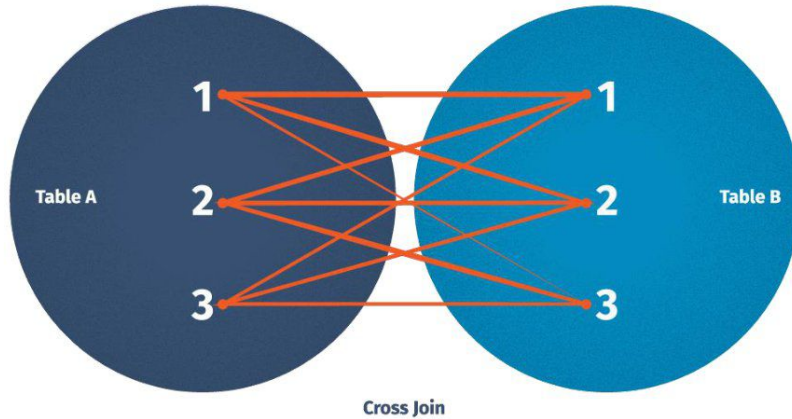
```
SELECT *
FROM Employee FULL JOIN DEPARTMENT
ON
Employee.dept_id=Department.dept_id;
```

Output

emp_id	emp_name	salary	dept_id	dept_id	dept_name
1	anish	25000	1	1	sales
2	sita	55000	2	2	marketing
3	ronit	40000	4	4	operations
4	riya	50000	5	NULL	NULL
Null	Null	Null	Null	3	finance

CROSS JOIN:

- ✓ Returns the Cartesian product of the two tables, which means it combines every row from the first table with every row from the second table.
- ✓ It does not require a join condition.
- ✓ Here each row is the combination of rows of both tables:



Syntax:

```
SELECT column(s) name  
FROM table1 CROSS JOIN table2;
```

Example:

```
SELECT *  
FROM Employee CROSS JOIN Department;
```

emp_id	emp_name	salary	dept_id	dept_id	dept_name
1	anish	25000	1	1	sales
2	sita	55000	2	1	sales
3	ronit	40000	4	1	sales
4	riya	50000	5	1	sales
1	anish	25000	1	2	marketing
2	sita	55000	2	2	marketing
3	ronit	40000	4	2	marketing
4	riya	50000	5	2	marketing
1	anish	25000	1	3	finance
2	sita	55000	2	3	finance
3	ronit	40000	4	3	finance
4	riya	50000	5	3	finance
1	anish	25000	1	4	operations
2	sita	55000	2	4	operations

3	ronit	40000	4	4	operations
4	riya	50000	5	4	operations

NATURAL JOIN

- ✓ The NATURAL JOIN is a type of join in SQL that automatically matches columns with the same name in the joined tables.
- ✓ It eliminates the need to specify the join condition explicitly.
- ✓ The resulting join will include only one instance of columns with the same name. It automatically eliminates duplicate columns from the join result.

Syntax:

```
SELECT column(s) name
FROM table1 NATURAL JOIN table2;
```

Example

```
SELECT *
FROM Employee NATURAL JOIN Department;
```

Output

dept_id	emp_id	emp_name	salary	dept_name
1	1	anish	25000	sales
2	2	sita	55000	marketing
4	3	ronit	40000	operations

It's important to note that the NATURAL JOIN relies on columns having the same name and data types in both tables.

Stored procedure

- ✓ A stored procedure is a named collection of SQL statements that are precompiled and stored in a database.
- ✓ If the user has an SQL query that you write over and over again, keep it as a stored procedure and execute it.
- ✓ Users can also pass parameters to a stored procedure so that the stored procedure can act based on the parameter value that is given.
- ✓ Based on the statements in the procedure and the parameters we pass, it can perform one or multiple DML operations on the database, and return value, if any.
- ✓ Thus, it allows us to pass the same statements multiple times, thereby, enabling reusability.

Advantages

- ✓ **Reusability:** Once a stored procedure is created, it can be called multiple times from different parts of an application or by multiple users.
- ✓ **Improved performance:** Since stored procedures are precompiled and stored in the database, they can execute faster than sending individual SQL statements from an application to the database server. This is because the database server doesn't have to re-parse and optimize the code each time it is executed.
- ✓ **Reduced network traffic:** The server only passes the procedure name and parameter instead of the whole query, reducing network traffic.
- ✓ **Easy to modify:** WE can easily change the statements in a stored procedure as per necessary.
- ✓ **Security:** Stored procedures can provide an additional layer of security by allowing access to the underlying data through the procedure while restricting direct access to the tables.
- ✓ **Modularity and encapsulation:** Stored procedures enable the modularization and encapsulation of database logic, making it easier to maintain and update the database code.
- ✓ **Parameterization:** Stored procedures can accept input parameters, allowing you to pass values into the procedure at runtime. These parameters can be used within the SQL statements to make the procedure more flexible and reusable

Creating stored procedure

To create a stored procedure in **MySQL**, we can use the following syntax:

```
DELIMITER //  
CREATE PROCEDURE procedure_name(parameter1 datatype, parameter2 datatype, ...)  
BEGIN  
    ----Statements using the input parametes  
END //  
DELIMITER ;
```

Note:

- ✓ **DELIMITER //** is used to change the delimiter temporarily so that you can use the semicolon ; within the procedure body without ending the entire statement prematurely.
- ✓ **DELIMITER ;** sets the delimiter back to the default semicolon ;

Executing stored procedure

To execute a stored procedure in MySQL, you can use the CALL statement followed by the name of the procedure and list of parameters if any. The syntax is as follows:

CALL procedure_name(parameter_list);

Creating stored procedure without parameters**Example:**

```
DELIMITER //  
CREATE PROCEDURE getallEmployee ()  
BEGIN  
SELECT * FROM employee;  
END //  
DELIMITER ;
```

Now, we can execute the above stored procedure as follows

```
CALL getallEmployee();
```

Creating parameterized stored procedure

In SQL, a parameterized procedure is a type of stored procedure that can accept input parameters. These parameters can be used to customize the behavior of the procedure and perform operations based on the input values provided.

To create a parameterized stored procedure in MySQL, we can define input parameters within the procedure definition.

Example:

```
DELIMITER //  
CREATE PROCEDURE getdepartmentEmployee (dept varchar(30))  
BEGIN  
SELECT *  
FROM employee  
WHERE department=dept;  
END //  
DELIMITER ;
```

To call this stored procedure, you can use the CALL statement and pass the parameter value:

```
CALL getdepartmentEmployee('civil');
```

Drop procedure

We can use the DROP PROCEDURE statement followed by the name of the procedure.

Here's the syntax:

```
DROP PROCEDURE procedure_name;
```

Example:

```
DROP PROCEDURE getdepartmentEmployee;
```

Views

- ✓ In SQL, a view is a virtual table based on the result-set of an SQL statement.
- ✓ A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.
- ✓ Views are defined based on queries, and they can be used to simplify complex queries, restrict access to certain data, or present a customized perspective of the data to different users or applications.

Syntax

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Types of views:

1. **simple view:** creating views from single table
2. **complex view:** creating views from multiple table

For dropping view

Syntax: DROP VIEW view_name;

Need of views

In some cases, it is not desirable for all users to see all the actual relations stored in the database.

For example consider the following relation

Employee(emp_id,emp_name,postion,salary,dept_id)
Department(dep_id,dept_name,location,budget)

Consider a person who needs to know information of employees with name, position and department name but not salary as well as other information, then this person should see a relation described. In such case views are created.

Example

```
CREATE VIEW employee_info as
SELECT Employee.emp_name, Employee.position, Department.dept_name
FROM Employee,Department
WHERE Employee.dept_id=Department.dept_id;
```

Now we can deny direct access to the Employee and Department relation but grant access to the view employee_info which consists of only attributes emp_name, position from relation Employee and another attribute dept_name from Department .

SQL OLD QUESTIONS SOLUTION

1. Write SQL statements for the following queries in reference to relation Emp_time provided. [PU:2010 Spring,PU:2011 fall]

Eid#	Name	Start_time	End_time
E101	Magale	10:30	18:30
E102	Malati	8:30	14:30
E103	Fulmaya	9:00	18:00

- i)create the table Eid# as primary key and insert the values provided

```
CREATE TABLE Emp_time (  
  `Eid#` VARCHAR(10) PRIMARY KEY,  
  Name VARCHAR(30),  
  Start_time TIME,  
  End_time TIME  
);
```

```
INSERT INTO Emp_time VALUES ('E101','Mangale', '10:30:00', '18:30:00');  
INSERT INTO Emp_time VALUES ('E102','Malati', '8:30:00', '14:30:00');  
INSERT INTO Emp_time VALUES ('E103', 'Fulmaya', '9:00:00', '18:00:00');
```

Note: using backticks around the column name will allow us to use special characters. In above example we are using backtick in column name Eid#. Here special character # is used. So column name becomes `Eid#`

- ii)Display the name of the employee whose name start from letter 'M' and who work more than seven hours

```
SELECT Name  
FROM Emp_time  
WHERE Name LIKE 'M%' AND TIMEDIFF(End_time, Start_time) > '07:00:00';
```

- iii)Delete the entire content of the table so that new records can be inserted.

```
TRUNCATE TABLE Emp_time;
```

2. Consider the following relational database

Employee(Empno,Name,Address)

Project(Pno,Pname)

Workon(Empno,Pno)

Parts(Partno,Partname,Qty_on_Hand)

Use(Empno,Pno,Partno)

The primary key are underlined

Now write SQL command for the following:

- i) Modify the database so that Jones now lives in Pokhara

```
UPDATE Employee
SET Address = 'Pokhara'
WHERE Name = 'Jones';
```

- ii) Give and SQL DDL definition for Employee table(assume your own data types for attributes)

```
CREATE TABLE Employee (
    Empno INT PRIMARY KEY,
    Name VARCHAR(30),
    Address VARCHAR(30)
);
```

- iii) Insert a new record into the employee table

```
INSERT INTO Employee
VALUES (11, 'Roshan Pokhrel', 'Kathamandu');
```

- iv) To retrieve the name of the employee who are working on a project named "DBMS"

```
SELECT Employee.Name
FROM Employee, Workon, Project
WHERE Employee.Empno = Workon.Empno
AND Workon.Pno = Project.Pno
AND Project.Pname = 'DBMS';
```

3. Consider the following relation: **[PU:2012 Spring]**

Employee(empID,FirstName,LastName,address,DOB,sex,position,deptNo)
Department(deptNo,deptName,mgr,empID)
Project(projNo,projName,deptNo)
Work on(empID,projNo,hour worked)

Write SQL statements for the following

i) List the name and addresses of all employees who works for IT department

```
SELECT CONCAT(Employee.FirstName, ' ', Employee.LastName) AS name, Employee.address
FROM Employee, Department
WHERE Employee.deptNo = Department.deptNo
AND Department.deptName = 'IT';
```

ii) List the total hours worked by each employee, arranged in order of department number and within department alphabetically by employee surname

```
SELECT Employee.empID, Employee.FirstName,Employee.LastName,Employee.deptNo,
SUM(`hour worked`) AS Totalhoursworked
FROM Employee, `Work on`
WHERE Employee.empID = `Work on`.empID
GROUP BY Employee.empID, Employee.FirstName, Employee.LastName, Employee.deptNo
ORDER BY Employee.deptNo, Employee.LastName;
```

iii) List the total number of employees in each department for those departments with more than 10 employees .

```
SELECT Department.deptName, COUNT(Employee.empID) AS TotalEmployees
FROM Employee, Department
WHERE Department.deptNo = Employee.deptNo
GROUP BY Department.deptName
HAVING COUNT(Employee.empID) > 10;
```

iv) List the project number ,project name and the number of employees who work in that project

```
SELECT Project.projNo, Project.projName, COUNT(`Work on`.empID) AS num_employees
FROM Project , `Work on`
WHERE Project.projNo = `Work on`.projNo
GROUP BY Project.projNo, Project.projName;
```

Note: using backticks around the column name will allow us to use characters with blank space. Here `Work on` can be used.

4. Consider the relational database [PU:2012 fall]
Employee(emp_name,street,city)
Works(empname,cmpname,salary)
Company(cmpname,city)
Manages(empname,cmpname)

Write SQL statement to:

- i. **Modify the database so that Amrit now lives in Naxal**

```
UPDATE Employee
SET city = 'Naxal'
WHERE emp_name = 'Amrit';
```

- ii. **Delete all tuples in the works relation for employee of xyz corporation**

```
DELETE FROM Works
WHERE cmpname = 'xyz corporation';
```

- iii. **Increase salary of all employees of ABC company by 10%**

```
UPDATE Works
SET salary = salary * 1.1
WHERE cmpname = 'ABC';
```

- iv. **Display all company name located at city pokhara or kathmandu from company tables**

```
SELECT cmpname
FROM Company
WHERE city = 'Pokhara' OR city= 'Kathmandu';
```

- v. **Display all empname who have salary greater than 5000 from works table**

```
SELECT empname
FROM Works
WHERE salary > 5000;
```

5. Consider the table tbl_emp as follow

[PU:2013 spring]

EmpId*	EmpName	Salary(Nrs.)	Date_of_join	Phone	Department
E001	Ram	20000	2060-02-01	#1234	Packing
E002	Hari	18000	2065-04-01	#5647	Cleaning
E004	Sita	15000	2068-04-01	#2564	Polishing

Write the SQL statements for the following

i) **Insert a new record**

```
INSERT INTO Employee  
VALUES ('E004', 'Gita', 22000, '2070-08-15', '#7890', 'Sales');
```

ii) **Delete the record of sita**

```
DELETE FROM Employee  
WHERE EmpName='Sita';
```

iii) **Change the Department of Hari to marketing**

```
UPDATE Employee  
SET Department = 'Marketing'  
WHERE EmpName= 'Hari';
```

iv) **Add a new column Address to the above table**

```
ALTER TABLE Employee  
ADD COLUMN Address VARCHAR(30);
```

v) **Increase the salary of all employee by 5000**

```
UPDATE Employee  
SET Salary = Salary + 5000;
```

vi) **Select the row having salary greater than 16000**

```
SELECT *  
FROM Employee  
WHERE Salary > 16000;
```

vii) **Delete the entire table**

```
DROP TABLE Employee;
```

6. Consider the relational database of figure given below, where primary keys are underlined.
Given an expression in SQL for each of the following queries. **[PU:2014 spring]**

Employee(employee_name,street,city)

Works(employee_name,company_name,salary)

Company(company_name,city)

Manages(employee_name,manager_name)

- i. **modify the databases so that Ram now lives in kathmandu**

```
UPDATE Employee
SET city = 'Kathmandu'
WHERE employee_name = 'Ram';
```

- ii. **Give all employees of First Bank Corporation a 10 percent raise**

```
UPDATE Works
SET salary = salary * 1.1
WHERE company_name = 'First Bank Corporation';
```

- iii. **Give all managers of First Bank Corporation a 10 percent raise**

```
UPDATE WORKS
SET salary = salary * 1.1
WHERE employee_name IN (SELECT manager_name FROM Manages)
AND company_name = 'First Bank Corporation';
```

- iv. **Delete all in the work relation for employees of small bank corporation**

```
DELETE FROM Works
WHERE company_name = 'Small Bank Corporation';
```

- v. **Find all employees who earn more than the average salary of all employees of their company**

```
SELECT employee_name
FROM Works a
WHERE salary > (
    SELECT avg(salary)
    FROM Works b
    WHERE a.company_name = b.company_name
);
```

7. Consider the following relations: **[PU: 2014 fall]**

Employee(emp_name,street,city)

Works(emp_name,company,salary)

Company(comp_name,city)

Manages(emp_name,manager_name)

Write SQL statements for:

i. Find employee names that lives in the city same as the company city

```
SELECT Employee.emp_name
FROM Employee, Works, Company
WHERE Employee.emp_name = Works.emp_name
AND Works.company = Company.comp_name
AND Employee.city = Company.city;
```

ii. List all the employee details who earn more than 25000

```
SELECT Employee.emp_name,Employee.street,Employee.city
FROM Employee, Works
WHERE Employee.emp_name = Works.emp_name
AND Works.salary > 25000;
```

iii. Update address of an employee 'Sriyash' to 'Pokhara'

```
UPDATE Employee
SET city = 'Pokhara'
WHERE emp_name = 'Sriyash';
```

iv. Create view for employee earns RS. 20,000 or more

```
CREATE VIEW HighEarningEmployee AS
SELECT Employee.emp_name,Employee.street,Employee.city,Works.salary
FROM Employee, Works
WHERE Employee.emp_name = Works.emp_name
AND Works.salary >= 20000;
```

v. Delete all the employees from the table employee

```
DELETE FROM Employee;
```

8. Suppose we are given the following table definitions with certain records in each table.

[PU: 2015 Spring]

EMPLOYEE(EID,NAME,POST,AGE)

POST(POST_TITLE,SALARY)

PROJECT (PID,PNAME,DURATION,BUDGET)

WORK-IN (PID,EID,JOIN_DATE)

Write SQL statements for

i) List the name of Employees whose age is greater than average age of all the employees

```
SELECT NAME
FROM EMPLOYEE
WHERE AGE > (
  SELECT AVG(AGE)
  FROM EMPLOYEE
);
```

ii) Display all the employee numbers of those employees who are not working in any project

```
SELECT EID
FROM EMPLOYEE
WHERE EID NOT IN (
  SELECT EID
  FROM WORK_IN
);
```

iii) List the name of employee and their salary who are working in the project 'DBMS'

```
SELECT EMPLOYEE.NAME, POST.SALARY
FROM EMPLOYEE, POST,PROJECT,WORK_IN
WHERE PROJECT.PID=WORK_IN.PID AND
WORK_IN.EID=EMPLOYEE.EID AND
EMPLOYEE.POST=POST.POST_TITLE AND
PROJECT.PNAME='DBMS';
```

iv) update the database so that "Rishab now lives in "Butwal"

(This question cannot be solved from above relation. It seems question is incorrect)

9. Write SQL statements for the following queries in reference to relation Emp_time provided. [2015 Fall]

Eid*	Name	Start_time	End_time
E101	Hari	10:15	18:00
E102	Malati	8:00	15:30
E103	Kalyan	9:30	17:00

- i) create the table Eid* as primary key and insert the values provided.
- ii) Display the name of the employee whose name starts from letter 'M' and who work more than seven hours.
- iii) Delete the entire content of the table so that new records can be inserted.

Solution:

(similar to Question of PU:2010 Spring and solution is already done in Question No.1)

10. Consider a simple relational database of Hospital management system.(Underlined attributes represent primary key attributes)

Doctors (Doctor_ID, DoctorName, Department, Address, Salary)

Patients(PatientID, Patient_Name, Address, Age, Gender)

Hospitals (PatientID, DoctorID, HospitalName, Location)

Write down the SQL statements for the following

[PU:2016 spring][PU:2019 fall]

- i. Display ID of patient admitted in hospital at pokhara and whose name ends with 'a'**

```
SELECT Patients.PatientID
FROM Patients, Hospitals
WHERE Hospitals.PatientID = Patients.PatientID
AND Hospitals.Location= 'Pokhara' AND Patients.Patient_Name LIKE '%a';
```

- ii. Delete the records of Doctors whose salary is greater than average salary of doctors**

```
DELETE FROM Doctors
WHERE Salary > (SELECT AVG(Salary) FROM Doctors);
```

- iii. Increase salary of doctors by 18.5% who works in OPD department**

```
UPDATE Doctors
SET Salary = Salary * 1.185
WHERE Department = 'OPD';
```

- iv. Find the average salary of Doctors for each address who have average salary more than 55k.**

```
SELECT Address, AVG(Salary) AS AverageSalary
FROM Doctors
GROUP BY Address
HAVING AVG(Salary) > 55000;
```

11. Consider the relational schema:

[PU:2016 fall]

Teacher (TeacherID,TeacherName,Office)

Write SQL statements for the following task:

i. To create a table from a table

```
CREATE TABLE Teacher
(
  TeacherID INT PRIMARY KEY,
  TeacherName VARCHAR(30),
  Office VARCHAR(30)
);
```

ii. To eliminate duplicate rows

```
SELECT DISTINCT *
FROM Teacher;
```

iii. To add new column 'Gender' in the table

```
ALTER TABLE Teacher ADD Gender VARCHAR(10);
```

(Note: This query is for mariadb)

iv. To sort data in a table

```
SELECT *
FROM Teacher
ORDER BY TeacherName ASC;
```

v. To delete rows

```
DELETE FROM Teacher
WHERE TeacherID = 123;
```

In this example, rows with TeacherID equal to 123 is deleted. we can modify the condition in the WHERE clause to match your specific deletion criteria.

vi. Count the number of rows based in office

```
SELECT Office, COUNT(*) AS RowCount
FROM Teacher
GROUP BY Office;
```

12. Write the SQL statements for the following Queries by reference to Liquors_info relation:

[PU:2017 fall]

Serial_No	Liquors	Start_year	Bottles	Ready_Year
1	Gorkha	1997	10	1998
2	Divine Wine	1998	5	2000
3	Old Durbar	1997	12	2001
4	Khukhuri Rum	1991	10	1992
5	Xing	1994	5	1995

i) creates the Liquors_info relation

```
CREATE TABLE Liquors_info
(
    Serial_No INT,
    Liquors VARCHAR(50),
    Start_year YEAR,
    Bottles INT,
    Ready_Year YEAR
);
```

ii)insert the records in Liquor_info as above

```
INSERT INTO Liquors_info
VALUES (1, 'Gorkha', '1997', 10, '1998');
```

```
INSERT INTO Liquors_info
VALUES (2, 'Divine Wine', '1998', 5, '2000');
```

```
INSERT INTO Liquors_info
VALUES (3, 'Old Durbar', '1997', 12, '2001');
```

```
INSERT INTO Liquors_info
VALUES (4, 'Khukhuri Rum', '1991', 10, '1992');
```

```
INSERT INTO Liquors_info
VALUES (5, 'Xing', '1994', 5, '1995');
```

iii)List all the records which were ready by 2000

```
SELECT *
FROM Liquors_info
WHERE Ready_Year <= '2000';
```

iv)Remove all records from database that required more than 2 years to get ready

```
DELETE FROM Liquors_info
WHERE (Ready_Year - Start_year) > 2;
```

13. Write SQL statements for the following:

[PU:2018 spring]

create a table named **Vehicle** with **veh_number** as primary key and following attributes:

veh_type,veh_brand,veh_year,veh_mileage,veh_owner,veh_photo,veh_price

```
CREATE TABLE Vehicle (  
  veh_number VARCHAR(50) PRIMARY KEY,  
  veh_type VARCHAR(50),  
  veh_brand VARCHAR(50),  
  veh_year YEAR,  
  veh_mileage int,  
  veh_owner VARCHAR(50),  
  veh_photo LONGBLOB,  
  veh_price DECIMAL(12, 2)  
);
```

ii)Enter a full detailed information of a vehicle

```
INSERT INTO Vehicle  
VALUES ('B DE 5425', 'Sedan', 'Toyota', 2022, 55, 'Hari Pokhrel',  
LOAD_FILE('C:\\Users\\Downloads\\veh1.jpg'), 18500.75);
```

iii)Increment a vehicle price by 10,000

```
UPDATE Vehicle  
SET veh_price = veh_price + 10000  
WHERE veh_number = 'B AA 8422'
```

This SQL UPDATE statement will find the vehicle with veh_number B AA 8422 in the "Vehicle" table and increase its veh_price by 10,000.

iv)Remove all vehicle's records whose brand contains character 'o' second position

```
DELETE FROM Vehicle  
WHERE veh_brand LIKE '_o%';
```

v)Display the total price of all vehicles

```
SELECT SUM(veh_price) AS total_price  
FROM Vehicle;
```

vi)create a view a from above table

```
CREATE VIEW Low_price_vehicle AS  
SELECT veh_number, veh_type, veh_brand, veh_year, veh_price  
FROM Vehicle  
WHERE veh_price < 150000;
```

vii) Display details of vehicles ordering on descending manner in brand and by mileage when brand matches

```
SELECT *  
FROM Vehicle  
ORDER BY veh_brand DESC, veh_mileage DESC;
```

viii) change data type of year to datetime

```
ALTER TABLE Vehicle  
MODIFY COLUMN veh_year DATETIME;
```

14. Consider the following three relations

[PU:2018 fall]

Doctor(Name,age,address)

Works(Name,Depart_no,salary)

Department(Depart_no,dept_name,floor,room)

Write down the SQL statement for the following

i) Display the name of doctor who do not work in any department

```
SELECT Name  
FROM Doctor  
WHERE Name NOT IN (  
    SELECT Name  
    FROM Works  
);
```

ii) Modify the database so that Dr.Hari Lives in pokhara

```
UPDATE Doctor  
SET address = 'Pokhara'  
WHERE Name = 'Dr. Hari';
```

iii) Delete all records of Doctor working OPD department

```
DELETE FROM Doctor  
WHERE Name IN  
( SELECT Works.Name  
  FROM Works ,Department  
  WHERE Works.Depart_no = Department.Depart_no  
    AND Department.dept_name = 'OPD'  
);
```

iv) Display the name of doctors who works in at least two department

```
SELECT Doctors.Name  
FROM Doctor,Works  
WHERE Doctors. Name = Works.Name  
GROUP BY Doctors.Name  
HAVING COUNT(DISTINCT Works.Depart_no) >= 2;
```

15. Write the SQL statements for the following queries by reference of Hotel_details relation. [PU:2019 spring]

Hotel_id	Hotel_name	Estb_year	Hotel_star	Hotel_worth
1	Hyatt	2047	Five	15M
2	Hotel ktm	2043	Three	5M
3	Fullbari	2058	Five	20M
4	Yak and Yeti	2052	Four	11M
5	Hotel chitwan	2055	Three	7M

i) create a database named hotel and table relation

-- Create the database

```
CREATE DATABASE hotel;
```

-- Switch to the hotel database

```
USE hotel;
```

-- Create the table Hotel_details

```
CREATE TABLE Hotel_details (  
    Hotel_id INT,  
    Hotel_name VARCHAR(50),  
    Estb_year INT,  
    Hotel_star VARCHAR(10),  
    Hotel_worth BIGINT  
);
```

ii) create a view named price which shows hotel name and its worth

```
CREATE VIEW price AS  
SELECT Hotel_name, Hotel_worth  
FROM Hotel_details;
```

iii) modify the data so that hotel chitwan is now four star level

```
UPDATE Hotel_details  
SET Hotel_star = 'Four'  
WHERE Hotel_name = 'Hotel_chitwan';
```

iv) delete the records of all hotels having worth more than 9M

```
DELETE FROM Hotel_details  
WHERE Hotel_worth > 9000000;
```

16. Write SQL statement for the following schemas (underline indicates primary key)
[PU:2020 spring]

Employee(Emp_No,Name,Address)
Project(PNo,Pname)
Workon(Emp_No,PNo)
Part(Partno,Part_name,Qty_on_hand)
Use(Emp_No,PNo,Partno,Number)

a. Listing all the employee details who are not working yet

```
SELECT Emp_No, Name, Address
FROM Employee
WHERE Emp_No NOT IN (SELECT DISTINCT Empno FROM Workon);
```

b. Listing Part_name and Qty_on_hand those were used in DBMS project

```
SELECT Part.Part_name, Part.Qty_on_hand
FROM Part,Use,Project
WHERE Part.Partno = Use.Partno
AND Use.PNo=Project.PNo
AND Project.pname='DBMS';
```

c. List the name of the projects that are used by employee from London

```
SELECT DISTINCT Project.Pname
FROM Project,Use,Employee
WHERE Project.PNo = Use.PNo AND
Use.Emp_No = Employee.Emp_No AND
Employee.Address = 'London';
```

d. Modify the database so that Jones now lives in 'USA'

```
UPDATE Employee
SET Address = 'USA'
WHERE Name = 'Jones';
```

e. Update address of an employee 'Japan' to 'USA'

```
UPDATE Employee
SET Address = 'USA'
WHERE Address = 'Japan';
```

17. Write a SQL statements for the following [PU:2020 fall]

i) Create a table named Automotor with chasis_number as primary key and following attributes.

veh_brand, veh_name, veh_model, veh_year, veh_cost, veh_color, veh_weight

```
CREATE TABLE Automotor (  
  chasis_number INT PRIMARY KEY,  
  veh_brand VARCHAR(50),  
  veh_name VARCHAR(50),  
  veh_model VARCHAR(50),  
  veh_year YEAR,  
  veh_cost DECIMAL(10,2),  
  veh_color VARCHAR(10),  
  veh_weight DECIMAL(10,2)  
);
```

ii) Enter a full detailed information of automotor

```
INSERT INTO Automotor  
  
VALUES (1654, 'Toyota', 'Corolla', 'XE', 2020, 25000.00, 'Red', 1200.50);
```

iii) Change any Automotor's year to 2019

```
UPDATE Automotor  
SET veh_year = 2019  
WHERE chasis_number=125;
```

iv) Remove all Automotor's records whose model contains 'i' in last position

```
DELETE FROM Automotor  
  
WHERE veh_model LIKE '%i';
```

v) Display the total cost of all vehicles of the table Automotor

```
SELECT SUM(veh_cost) AS total_cost  
FROM Automotor;
```

vi) Create a view from above table having vehicle only red color

```
CREATE VIEW RedVehicles AS  
SELECT * FROM Automotor  
WHERE veh_color = 'red';
```

vii) Display details of Automotor ordering on descending manner by brand name and ascending order on model when brand matches

```
SELECT *  
FROM Automotor  
ORDER BY veh_brand DESC, veh_model ASC;
```

viii) change data type of veh_color so that it only takes one character

```
ALTER TABLE Automotor  
MODIFY COLUMN veh_color CHAR(1);
```

(Note: This query is for mariaDB. Syntax may vary in different DBMS)

18. Let us consider the following relation

Sailors (sid,sname,rating,age)
Boats(bid, bname,color)
Reserves(sid,bid,day)

Write a SQL statements for the following **[PU:2021 spring]**

i)Find the records of sailors who have reserved boat number 103(bid=103)

```
SELECT Sailors.sid,Sailors.sname,Sailors.rating,Sailors.age  
FROM Sailors , Reserves  
WHERE Sailors.sid = Reserves.sid  
AND Reserves.bid = 103;
```

ii)Update the color of the boat ,where bid is 104,into green

```
UPDATE Boats  
SET color = 'green'  
WHERE bid = 104;
```

iii) find the name of sailors who have reserved a red or green boat

```
SELECT Sailors.sname  
FROM Sailors, Reserves, Boats  
WHERE Sailors.sid = Reserves.sid  
AND Reserves.bid = Boats.bid  
AND Boats.color IN ('red', 'green');
```

iv) find the name of sailors who have reserved boat number 103 on day 5

```
SELECT Sailors.sname
FROM Sailors , Reserves
WHERE Sailors.sid = Reserves.sid
AND Reserves.bid = 103
AND Reserves.day = 5;
```

v) find the name of sailors whose name is not 'Ram'

```
SELECT sname
FROM Sailors
WHERE sname != 'Ram';
```

vi) find the name of all boats

```
SELECT bname
FROM Boats;
```

18. Consider the relation Actress_Details and Write SQL statements for the following queries.
[PU:2022 fall]

Players_id	Actress_name	Debut_year	Recent_release	Actress_fee
1	Renu	2010	Samay	400000
2	Sita	2022	Radha	300000
3	Geeta	2001	Mato	600000
4	Amita	1990	Man	700000
5	Karishma	1989	Prem	100000

i) Create the table Actress_details relation

```
CREATE TABLE Actress_details (
    Players_id INT PRIMARY KEY,
    Actress_name VARCHAR(50),
    Debut_year YEAR,
    Recent_release VARCHAR(50),
    Actress_fee INT
);
```

ii) Delete the data of actress whose recent release is prem

```
DELETE FROM Actress_details
WHERE Recent_release = 'Prem';
```

- iii) Modify the database so that Renu's new release is "Win the race" film

```
UPDATE Actress_details
SET Recent_release = 'Win the race'
WHERE Actress_name = 'Renu';
```

- iv) Insert a new record in the above table

```
INSERT INTO Actress_details
VALUES (6, 'Priya', '2015', 'Sunset', 500000);
```

19. Write SQL statements for the following queries using the given Employees relation: [PU:2023 spring]

E_id	Fname	Lname	Department	Salary	Hire_Date
01	Ramu	Bashyal	Sales	20000	2023-08-08
02	Damu	Pandey	IT	50000	2022-01-01
03	Biru	B.k.	Sales	40000	2021-02-10
04	Hiru	Dhamala	HR	35000	2023-12-18
05	Biren	Khadka	IT	60000	2012-10-22

- i) Create a database named Company and Employees relation.**

```
CREATE DATABASE Company;
```

```
CREATE TABLE Employees(
E_id INT PRIMARY KEY,
Fname varchar(30),
Lname,varchar(30),
Department varchar(30),
Salary INT,
Hire_Date DATE
);
```

- ii) Create a view that shows the E_id ,Department and Hire_Date of all employees**

```
CREATE VIEW emp_view
SELECT E_id,Department,Hire_Date
FROM employees;
```

- iii) Modify the table such that the Department of Biren is HR now.**

```
Update Employees
SET Department='HR'
WHERE Fname='Biren';
```

- iv) Delete the record of employees whose Lname is "Pandey"**

```
DELETE FROM Employees
WHERE Lname='Pandey';
```

20. Consider the following relation

```
Orders(order_id,product_name,price,quantity,order_date,delivery_date)
```

1) Create table orders

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    product_name VARCHAR(50) ,  
    price DECIMAL(10, 2) ,  
    quantity INT ,  
    order_date DATE ,  
    delivery_date DATE  
);
```

2) Now insert any 8 records

```
INSERT INTO Orders  
VALUES (1, 'T-shirt', 25.99, 2, '2023-07-15', '2023-07-25');
```

```
INSERT INTO Orders  
VALUES (2, 'Jeans', 49.95, 1, '2023-07-17', '2023-07-20');
```

```
INSERT INTO Orders  
VALUES (3, 'Shoes', 69.50, 1, '2023-07-20', '2023-07-30');
```

```
INSERT INTO Orders  
VALUES (4, 'Sunglasses', 12.75, 3, '2023-07-22', '2023-07-28');
```

```
INSERT INTO Orders  
VALUES (5, 'Backpack', 34.99, 2, '2023-07-25', '2023-07-29');
```

```
INSERT INTO Orders  
VALUES (6, 'Headphones', 59.99, 1, '2023-07-29', '2023-08-05');
```

```
INSERT INTO Orders  
VALUES (7, 'Smartphone', 299.99, 2, '2023-07-29', '2023-11-01');
```

```
INSERT INTO Orders  
VALUES (8, 'Laptop', 799.95, 1, '2023-07-29', '2025-08-01');
```

3) Retrieve all orders placed on a 2023-07-15

```
SELECT *  
FROM Orders  
WHERE order_date = '2023-07-15';
```

4) Find the number of days that required to delivered shoes

```
SELECT DATEDIFF(delivery_date, order_date) AS delivery_time
FROM Orders
where product_name='shoes';
```

5) Find all the orders that is received from '2023-07-15' to '2023-07-25'

```
SELECT *
FROM Orders
WHERE order_date BETWEEN '2023-07-15' AND '2023-07-25';
```

6) find all the orders that is received today

```
SELECT *
FROM Orders
WHERE order_date = CURDATE();
```

7) Calculate the average number of days it takes to deliver a orders

```
SELECT AVG(DATEDIFF(delivery_date, order_date)) AS avg_delivery_time
FROM Orders;
```

Here, in DATDIFF() function we have passed two parameters that is **DATEDIFF(date1, date2)**

This will returns date difference in terms of number of days (date1-date2) and **this result occurs if we run this query on MySQL DBMS.**

But sometimes it is necessary to find out date difference in terms of number of month, week, year, quarter etc. In such case three parameters need to passed three parameters.

Syntax

```
DATEDIFF(interval, date1, date2)
```

Parameter Values

Parameter	Description
<i>interval</i>	Required. The part to return. Can be one of the following values: • year, yyyy, yy = Year • quarter, qq, q = Quarter • month, mm, m = month • dayofyear = Day of the year • day, dy, y = Day • week, ww, wk = Week • weekday, dw, w = Weekday • hour, hh = hour • minute, mi, n = Minute • second, ss, s = Second • millisecond, ms = Millisecond
<i>date1, date2</i>	Required. The two dates to calculate the difference between

Note:

- DATEDIFF() function with two parameters are supported in MYSQL DBMS.
- DATEDIFF() function with three parameters are supported in MS SQL Server DBMS

If you want test query in different DBMS ,you can follow this link

<http://sqlfiddle.com>

8) Find the number of months required to deliver smartphone

```
SELECT DATEDIFF(delivery_date, order_date) AS delivery_time
FROM Orders
where product_name='smartphone';
```

If we run this query in MySQL DBMS.

DATEDIFF() function will returns 95 for this query by considering above relations.

delivery_time
95

DATEDIFF() with three parameters are not supported in MySQL DBMS.

This query can be re-written as follows by passing three parameters in MS SQL server DBMS.

```
SELECT DATEDIFF(month, order_date,delivery_date) AS delivery_time
FROM Orders
where product_name='smartphone';
```

Note: you can write DATEDIFF() function with three parameters in exam.

9) Find the number of weeks required to deliver smartphone

```
SELECT DATEDIFF(week, order_date,delivery_date) AS delivery_time
FROM Orders
where product_name='smartphone';
```

10) Find the products that required more than 2 month to delivered

```
SELECT product_name
FROM orders
WHERE DATEDIFF(month,order_date,delivery_date)>2;
```

11) Find the products that required more than 3 weeks to delivered

```
SELECT product_name
FROM orders
WHERE DATEDIFF(week,order_date,delivery_date)>3;
```

12. Find the products that required more than 1 years to delivered.

```
SELECT product_name
FROM orders
WHERE DATEDIFF(year,order_date,delivery_date)>1;
```

The SQL SELECT TOP Clause

- ✓ The SELECT TOP clause is used to specify the number of records to return.
- ✓ The SELECT TOP clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

Note: Not all database systems support the SELECT TOP clause. MySQL supports the LIMIT clause to select a limited number of records, while Oracle uses FETCH FIRST n ROWS ONLY and ROWNUM.

My SQL syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

Find the top 3 records from orders from orders

```
SELECT *
FROM orders
LIMIT 3;
```

Create relational database for the Department of computer Engineering (DOCE) of pokhara university. Your database should have at least three relations Describe referential integrity constraint based on above database of DOCE.[PU:2017 spring]

Based on the Department of Computer Engineering (DOCE) of Pokhara University, we can create a relational database with following relations: "Student," "Faculty," "Course" "Enroll"

Student(student_id, student_name, email, address)
Faculty(faculty_id, faculty_name, qualification)
Course(course_id, course_name, course_description, faculty_id)
Enroll(enroll_id, student_id, course_id, enrollment_date)

Now creating tables

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50),  
    email VARCHAR(50),  
    address VARCHAR(100)  
);  
CREATE TABLE Faculty (  
    faculty_id INT PRIMARY KEY,  
    faculty_name VARCHAR(50),  
    qualification VARCHAR(50)  
);  
CREATE TABLE Course (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50),  
    course_description LONGTEXT,  
    faculty_id INT,  
    FOREIGN KEY (faculty_id) REFERENCES Faculty (faculty_id)  
);  
CREATE TABLE Enroll (  
    enroll_id INT PRIMARY KEY,  
    student_id INT,  
    course_id INT,  
    enrollment_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Student(student_id),  
    FOREIGN KEY (course_id) REFERENCES Course(course_id)  
);
```

Based on the provided relations, the following are the foreign key integrity constraints that can be applied to maintain referential integrity:

Course table:

The faculty_id column in the Course table is a foreign key referencing the faculty_id column in the Faculty table. This ensures that the faculty_id value in the Course table must exist in the Faculty table.

Enroll table:

The student_id column in the Enroll table is a foreign key referencing the student_id column in the Student table. This ensures that the student_id value in the Enroll table must exist in the Student table.

The course_id column in the Enroll table is a foreign key referencing the course_id column in the Course table. This ensures that the course_id value in the Enroll table must exist in the Course table.

These foreign key constraints help maintain data integrity by enforcing the relationships between the tables. They prevent the insertion of invalid values that do not exist in the referenced tables, ensuring the consistency of the data across the relations.

Note: You can draw schema diagram for above relations as well.